

Proyecto 2. Modelado y Programación. Reporte de Proyecto

Pérez Pérez Iker Daniel

7 de mayo de 2026

Índice

1. Introducción	3
2. Objetivos	3
2.1. Objetivo General	3
2.2. Objetivos Específicos	3
3. Tecnologías y Herramientas	4
4. Diseño del Sistema	5
4.1. Arquitectura General	5
4.2. Modelo	6
4.3. Controlador	8
4.4. Vista	8
5. Conclusión	10

1. Introducción

Las bases de datos musicales permiten almacenar, organizar y consultar información relacionada con canciones, artistas, álbumes, géneros y listas de reproducción, facilitando la administración eficiente de grandes cantidades de información multimedia. En este tipo de sistemas, la base de datos actúa como el núcleo encargado de gestionar la persistencia y recuperación de la información, mientras que la aplicación proporciona al usuario herramientas para interactuar con dichos datos mediante consultas y operaciones de manipulación.

En este contexto es donde entra el segundo proyecto de la materia de Modelado y Programación, el cual consiste en implementar un sistema de gestión musical conectado a una base de datos relacional que contenga la información de archivos mp3 usando las etiquetas Id3v2, permitiendo realizar operaciones de consulta y administración de información relacionada con canciones, artistas (distinguiendo entre grupos y solistas) y álbumes mediante una interfaz gráfica intuitiva para el uso del usuario final.

Para lograr esto, se hizo uso del lenguaje de programación Rust, de SQLite como sistema gestor de base de datos relacional, de Cargo como sistema de construcción, de docker como herramienta de desarrollo y se utilizaron crates del lenguaje tales como rusqlite para la comunicación con la base de datos, walkdir para recorrer directorios dentro del sistema y poder implementar el minero, lofty para el manejo de metadatos y rodio para la reproducción de archivos mp3 (música), eso el lado del modelo. Por otra parte se utilizó GTK4 para la construcción de la interfaz gráfica en el lado de la vista.

El presente documento describe el diseño, implementación y resultados del sistema desarrollado, así como las decisiones tomadas durante el proceso y las posibles mejoras futuras.

2. Objetivos

2.1. Objetivo General

El objetivo general del proyecto es implementar un sistema de gestión musical conectado a una base de datos relacional que permita almacenar, consultar y administrar información relacionada con canciones, artistas y álbumes, además de poder reproducir las canciones que existan en la base de datos. Las características que, de manera sintetizada, queremos que cumpla el sistema son: la capacidad de realizar búsquedas y consultas musicales mediante el uso de un lenguaje de consulta que se traduzca a SQL, reproducir archivos de audio desde la aplicación, mostrar la información de manera organizada mediante una interfaz gráfica y mantener la persistencia de los datos mediante una base de datos robusta y estructurada. Para la interacción entre el usuario y el programa se quiere utilizar una interfaz gráfica de usuario (GUI) hecha en GTK que facilite la navegación y manipulación de la información musical de manera intuitiva y visual, sobre todo teniendo la vista en formato de tabla o en formato de imágenes.

2.2. Objetivos Específicos

- Diseñar la arquitectura del sistema tomando en cuenta el patrón de diseño MVC (que es la base de todo el proyecto).

- Utilizar el patrón de diseño DAO de manera eficiente para la comunicación del programa con la base de datos, de manera que el código no esté contaminado por todas partes de SQL.
- Desarrollar un lenguaje de consulta sencillo para el usuario, el cual sea traducido posteriormente a consultas SQL para interactuar con la base de datos.
- Implementar los algoritmos correspondientes para el análisis sintáctico y posterior compilación del lenguaje de consulta.
- Desarrollar una interfaz gráfica amigable e intuitiva que permita visualizar y consultar la información musical de manera eficiente al usuario.
- Integrar un reproductor de audio dentro de la aplicación para permitir la reproducción de archivos mp3 que correspondan a las canciones almacenadas en el sistema.
- Realizar pruebas de las consultas y operaciones sobre la base de datos para verificar el correcto funcionamiento del sistema y la integridad de la información.

3. Tecnologías y Herramientas

Para el desarrollo de este proyecto se utilizaron diversas tecnologías que ayudaron al cumplimiento de los objetivos impuestos.

- **Rust:** en este caso ya se mencionaron las características generales del language en el primer reporte y los motivos de su elección en primer lugar, por lo que, para no ser repetitivos, mencionaremos brevemente el porque de la elección en este caso. Se escogió este language (además de todas sus bondades previamente explicadas en el proyecto anterior) debido a que considero que en el pasado proyecto al ser un language totalmente nuevo para mí, no aproveché su rendimiento al máximo, por lo que quería seguir en la curva de aprendizaje del language y aprovechar mejor lo que tiene para ofrecer ahora que poseo experiencia previa en el uso de este language.
- **Rusqlite** es uno de los crates fundamentales utilizados en el desarrollo del proyecto, ya que proporciona la comunicación entre la aplicación escrita en Rust y la base de datos SQLite. **Rusqlite** es una biblioteca ligera y eficiente que permite ejecutar consultas SQL, manipular resultados y administrar transacciones de manera segura gracias al sistema de tipos y manejo de errores de Rust. Entre sus ventajas se encuentra la facilidad con la que integra SQLite directamente dentro de la aplicación sin necesidad de servidores externos, además de ofrecer una interfaz bastante flexible para trabajar con consultas parametrizadas, lo cual ayuda a prevenir errores y mejorar la seguridad de las operaciones sobre la base de datos.
- **Walkdir** es un crate utilizado para recorrer directorios y archivos del sistema de manera recursiva. En este proyecto fue especialmente útil para localizar automáticamente archivos musicales dentro de carpetas seleccionadas por el usuario. Una de las principales ventajas de **Walkdir** es su simplicidad y eficiencia al navegar estructuras de directorios complejas, permitiendo recorrer grandes cantidades de archivos de manera ordenada y segura. Además, abstrae gran parte de la complejidad asociada al manejo manual de rutas y subdirectorios dentro del sistema operativo.

- **Lofty** es una biblioteca especializada en la lectura y manipulación de metadatos de archivos de audio. Este crate permitió obtener información relevante de las canciones, como el título, artista, álbum o género directamente desde los archivos musicales. Entre las ventajas de **Lofty** destaca el soporte para múltiples formatos de audio y etiquetas de metadatos (en este caso utilizamos Id3v2), así como la facilidad con la que integra esta información dentro de estructuras de Rust, permitiendo manipularla de forma segura y eficiente.
- **Rodio** es el crate encargado de la reproducción de audio dentro de la aplicación. **Rodio** proporciona una interfaz relativamente sencilla para reproducir archivos de sonido utilizando las capacidades multimedia del sistema. Entre sus beneficios se encuentra la facilidad para integrar reproducción de audio dentro de programas escritos en Rust sin necesidad de trabajar directamente con APIs de bajo nivel. Además, permite manejar reproducción, pausado y control básico de audio de manera eficiente y portable entre distintos sistemas operativos.
- **GTK** es la biblioteca utilizada para construir la interfaz gráfica de usuario del proyecto. Gracias a **GTK** fue posible desarrollar una interfaz visual que permite al usuario interactuar con la base de datos musical de manera intuitiva y organizada. Entre las ventajas de esta biblioteca destaca la gran cantidad de componentes gráficos disponibles, tales como tablas, botones, cuadros de texto y vistas organizadas, además de su integración multiplataforma. Otro aspecto importante es que **GTK** permite separar relativamente bien la lógica de la interfaz de la lógica de negocio, favoreciendo una arquitectura más limpia y mantenible.
- **Docker** es una plataforma de código abierto que permite empaquetar, distribuir y ejecutar aplicaciones dentro de los llamados contenedores ligeros y portátiles. Esta herramienta fue necesaria para poder ejecutar el sistema en distintas computadoras sin la necesidad de tener instalado el lenguaje de programación Rust ni todas las dependencias necesarias, sino que, el contenedor ya incluía todo lo necesario para la correcta ejecución del programa.
- **Git** es el sistema de control de versiones predilecto en nuestros días, además de que es gratuito y de código abierto, y es que no se desarrolla un proyecto de mediana dificultad como el que aquí se presenta sin contar con un sistema de control de versiones. En este caso en específico no hay mejor herramienta que Git ya que es bastante intuitivo pensar el desarrollo de software como una línea del tiempo, lo cual es justo lo que hace Git. Complementario a Git, también se utilizó GitHub como repositorio remoto ya que su interfaz es bastante amigable con el usuario y ofrece una mayor cantidad de integraciones con herramientas de terceros, siendo una herramienta más sencilla de utilizar que por ejemplo GitLab.

4. Diseño del Sistema

4.1. Arquitectura General

El desarrollo del presente proyecto requirió el uso del patrón de arquitectura MVC (Modelo-Vista-Controlador), ya que la aplicación debía separar de manera clara la lógica de negocio, la interacción con la base de datos y la interfaz gráfica del usuario. Bajo este

enfoque, el sistema fue dividido en tres componentes principales: el modelo, encargado de representar y manipular la información musical, además de la interacción con la base de datos y el compilador para el lenguaje de consulta; la vista, responsable de mostrar la interfaz gráfica al usuario mediante GTK; y el controlador, cuya función consiste en coordinar las acciones entre la vista y el modelo, procesando las entradas del usuario y ejecutando las operaciones correspondientes.

Además del patrón MVC, también se utilizó el patrón DAO (Data Access Object) para abstraer completamente el acceso a la base de datos. De esta manera, toda la lógica relacionada con consultas SQL, inserciones, búsquedas y recuperación de información se encapsuló dentro de estructuras especializadas encargadas exclusivamente de la comunicación con SQLite mediante `rusqlite`. Gracias a esto, la lógica de la aplicación permanece desacoplada de los detalles específicos del manejo de la base de datos, favoreciendo una arquitectura más modular y mantenible.

También fue necesario incorporar un cuarto componente que actuara como punto de entrada para el proyecto el cual fue denominado aplicación.

Gracias a esta organización, el sistema quedó estructurado en módulos relativamente independientes, permitiendo separar claramente la interfaz gráfica, la lógica de negocio, el procesamiento de consultas y el acceso a los datos, lo cual facilitó considerablemente el desarrollo, mantenimiento y escalabilidad de la aplicación.

4.2. Modelo

El modelo es el núcleo principal de este proyecto, ya que concentra toda la lógica relacionada con el manejo de la información musical, el procesamiento de consultas y la comunicación con la base de datos. A través de este componente se gestionan las operaciones necesarias para recuperar, almacenar y manipular la información utilizada por la aplicación, todo esto por medio de un minero. Además, el modelo es el encargado de coordinar la traducción del lenguaje de consultas implementado en el sistema hacia consultas SQL válidas, así como de administrar la obtención de metadatos musicales y las operaciones relacionadas con las canciones. Por último, el modelo también se encarga de la reproducción de los archivos mp3. El módulo del modelo se divide en los siguientes componentes:

- **minero.rs**. Es uno de los módulos principales del modelo, ya que se encarga de recorrer directorios del sistema de archivos en busca de canciones en formato MP3 utilizando el crate `Walkdir`. Una vez encontrados los archivos, el módulo analiza sus metadatos mediante `Lofty`, extrayendo información relevante como título, artista, álbum, género, año y número de pista. Además, construye representaciones de alto nivel de los objetos musicales utilizados por la aplicación, tales como canciones, álbumes, artistas, grupos y personas. Este módulo también contiene la lógica necesaria para manejar información faltante dentro de los metadatos, proporcionando valores por omisión cuando algún atributo no puede obtenerse correctamente desde las etiquetas ID3v2 de los archivos de audio.
- **compilador.rs**. Es el módulo encargado de procesar el lenguaje de consultas diseñado para el sistema y traducirlo a instrucciones SQL válidas que puedan ejecutarse sobre la base de datos. Su función principal consiste en analizar las consultas escritas por el usuario, validar que respeten la sintaxis del lenguaje implementado y posteriormente generar las consultas SQL equivalentes para realizar búsquedas y

operaciones sobre la información musical almacenada. Este módulo abstrae completamente al usuario de la complejidad del lenguaje SQL, permitiendo interactuar con la base de datos mediante instrucciones más simples y controladas. Además, concentra gran parte de la lógica relacionada con el análisis léxico y sintáctico de las consultas, así como la construcción dinámica de sentencias SQL dependiendo de los filtros y condiciones especificadas por el usuario.

- **crea_entidades.rs**. Es el módulo encargado de transformar la información obtenida de los metadatos musicales en entidades de alto nivel que representan directamente los registros de la base de datos. Su función principal consiste en construir estructuras como **Performer**, **Album**, **Person** y **Group** a partir de los objetos de metadatos generados durante el proceso de minería de canciones. Este módulo abstrae la conversión entre la información extraída de los archivos MP3 y las entidades utilizadas por los DAOs para interactuar con la base de datos, facilitando considerablemente la separación entre la lógica de extracción de datos y la lógica de persistencia. Además, incorpora manejo de valores por omisión para atributos faltantes dentro de los metadatos, garantizando que las entidades creadas siempre contengan información válida para ser almacenada y manipulada dentro del sistema.
- **reproductor/reproductor.rs**. Es el módulo encargado de la reproducción de audio dentro de la aplicación utilizando el crate **Rodio**. Su función principal consiste en cargar y reproducir archivos musicales a partir de la ruta de una canción almacenada en la base de datos. Para ello, el módulo crea y administra un flujo de salida de audio del sistema, además de mantener un **Sink** encargado de controlar la reproducción de las canciones. Este componente abstrae gran parte de la complejidad relacionada con el manejo de audio de bajo nivel, permitiendo que la aplicación reproduzca archivos MP3 de manera relativamente sencilla. Además, encapsula toda la lógica relacionada con la decodificación de archivos de audio y su envío al dispositivo de salida correspondiente.
- **rola_vista.rs**. Es el módulo encargado de guardar la información de la canción que se le va a representar al usuario en la parte de la vista.
- **metadatos.rs**. Se encarga de guardar los metadatos de las etiquetas Id3v2 obtenidos durante el proceso de minado, sirven como paso intermedio entre la información cruda de las etiquetas y la información que se va a guardar en la base de datos.
- **manejador_dao.rs**. Es el módulo encargado de realizar todo el proceso de agregado de una canción a la base de datos después de ser minada, cumpliendo con todas las relaciones establecidas en el esquema de la base de datos.
- **entidades/album.rs**, **group.rs**, **performer.rs**, **person.rs**, **rola.rs**. Son structs encargadas de guardar la información de los campos de cada tabla de la base de datos para poder representar los datos a un nivel más alto.
- **dao/album_dao.rs**, **group_dao.rs**, **performer_dao.rs**, **person_dao.rs**, **rola_dao.rs**. Son las implementaciones del patron de diseño DAO para cada una de las tablas de la base de datos, cada una viene con su implementación para agregar, hacer búsquedas por cada uno de los campos de la tabla y eliminar de la tabla.

4.3. Controlador

El controlador es el componente encargado de coordinar la interacción entre la vista y el modelo dentro de la arquitectura MVC utilizada en el proyecto. Su función principal consiste en recibir las acciones realizadas por el usuario desde la interfaz gráfica, procesarlas y ejecutar las operaciones correspondientes sobre el modelo. De esta manera, el controlador actúa como intermediario entre la lógica de negocio y la interfaz visual de la aplicación.

- **controlador.rs.** Es el módulo principal del controlador dentro de la arquitectura MVC y uno de los componentes más importantes de toda la aplicación, ya que centraliza la comunicación entre la interfaz gráfica, el modelo y la base de datos. Este módulo se encarga de coordinar las operaciones principales del sistema, tales como la creación de la base de datos, la minería de archivos musicales, el almacenamiento de canciones en SQLite y la recuperación de información para ser mostrada en la vista.

Además, abstrae gran parte de la lógica de interacción con el modelo, permitiendo que la vista únicamente invoque métodos de alto nivel sin preocuparse por los detalles internos de acceso a datos o procesamiento de información. Para ello, utiliza estructuras como **ManejadorDao** y distintos DAOs especializados para manipular la información almacenada en la base de datos.

Otro aspecto importante de este módulo es que también coordina la reproducción de canciones desde la interfaz gráfica mediante el módulo **Reproductor**, encapsulando completamente la lógica relacionada con el audio. Asimismo, transforma las entidades recuperadas de la base de datos en estructuras simplificadas orientadas a la vista, facilitando la representación de la información dentro de tablas y componentes gráficos de GTK.

- **cancion_vista.rs.** Struct encargada de presentar la información de la canción que se requiere en la vista.

4.4. Vista

La vista es el componente encargado de toda la interacción visual con el usuario mediante una interfaz gráfica construida con GTK. Su función principal consiste en mostrar de manera organizada la información musical almacenada en la base de datos, así como proporcionar los componentes gráficos necesarios para que el usuario pueda realizar búsquedas, seleccionar canciones y ejecutar distintas acciones dentro de la aplicación. Además, la vista se encarga de representar tablas, listas y controles gráficos que facilitan la navegación y consulta de la información musical, buscando que la interacción con el sistema sea intuitiva y amigable para el usuario final. También administra eventos generados desde la interfaz gráfica, tales como selecciones de canciones, reproducción de audio y ejecución de consultas escritas por el usuario en el lenguaje implementado por el sistema.

- **ui.rs.** Es el módulo principal de la vista y el encargado de construir toda la interfaz gráfica de la aplicación utilizando GTK. Este módulo define la estructura visual completa del sistema, incluyendo la ventana principal, el panel del reproductor,

las vistas de biblioteca musical y los distintos componentes gráficos con los que interactúa el usuario.

Entre sus responsabilidades principales se encuentra la creación y organización de widgets como botones, tablas, vistas en cuadrícula, paneles redimensionables, áreas de texto y barras de reproducción. Además, administra la distribución visual de la interfaz mediante contenedores y componentes gráficos de GTK, buscando proporcionar una experiencia visual organizada y amigable para el usuario.

Otro aspecto importante de este módulo es la conexión de eventos entre la interfaz y el controlador. A través de distintos `callbacks`, el módulo responde a acciones del usuario como seleccionar canciones, cambiar entre vista de tabla y tarjetas, ejecutar consultas, reproducir canciones y ejecutar el proceso de minería de archivos musicales. Para ello, utiliza referencias compartidas al controlador, delegando completamente la lógica de negocio y acceso a datos al componente correspondiente de la arquitectura MVC.

- **widgets/biblioteca.rs.** Es el módulo encargado de administrar y representar visualmente la biblioteca musical dentro de la interfaz gráfica de la aplicación. Su función principal consiste en cargar, organizar y mostrar las canciones obtenidas desde la base de datos utilizando distintos componentes gráficos de GTK, tales como vistas en cuadrícula (`FlowBox`) y vistas tabulares (`ColumnView`).

Este módulo abstrae toda la lógica relacionada con la representación visual de las canciones, permitiendo construir tarjetas gráficas con información relevante como título, artista y álbum, además de administrar dinámicamente las canciones cargadas dentro de la interfaz. Para ello, mantiene internamente una colección compartida de canciones en formato `CancionVista`, facilitando el acceso y sincronización de la información mostrada al usuario.

Además, el módulo permite alternar entre distintas formas de visualización de la biblioteca musical, proporcionando una representación más flexible y amigable para el usuario final. Gracias a esta organización, la lógica relacionada con la presentación gráfica de las canciones permanece desacoplada del resto de componentes del sistema.

- **reproductor.rs.** Es el módulo de la vista encargado de gestionar la interacción entre la interfaz gráfica y el sistema de reproducción de audio. Su función principal consiste en administrar la canción actualmente seleccionada por el usuario y coordinar la ejecución de la reproducción mediante el controlador de la aplicación.

Este módulo encapsula la lógica relacionada con los controles gráficos del reproductor, particularmente el botón de reproducción y la etiqueta que muestra el nombre de la canción activa. A través de distintos `callbacks` conectados a eventos de GTK, el módulo responde a las acciones del usuario y delega la reproducción efectiva de la canción al controlador correspondiente.

Además, mantiene internamente una referencia compartida a la canción seleccionada dentro de la interfaz gráfica, permitiendo actualizar dinámicamente la información mostrada en el reproductor conforme el usuario selecciona distintas canciones dentro de la biblioteca musical. Gracias a esta organización, el módulo desacopla la lógica visual del reproductor de la lógica de reproducción de audio implementada dentro del modelo y el controlador.

5. Conclusión

En este proyecto se intentó desarrollar un sistema de gestión musical utilizando una base de datos relacional que implementara un proceso de búsqueda de canciones utilizando un language que posteriormente se compilara a SQL. Además de implementar un proceso de minado para poblar la base de datos y un reproductor musical de los archivos mp3 que pertenezcan a la base de datos.

Sin embargo, debido a una pésima planeación y ejecución de los tiempos que requerían las diversas partes de la implementación el sistema final carece de funcionalidades primarias como el compilador para el language requerido, además el reproductor está en una etapa muy temprana del desarrollo por lo que cuando se reproduce una canción no es posible reproducir otra hasta que la primera termine. Por otra parte el proceso de desarrollo careció de pruebas unitarias y de comentarios para documentación.

Por lo anterior, es claro que se debe mejorar la planificación en el proceso de implementación de las diversas características del sistema y aprender a saber reconocer las funcionalidades prioritarias que deberían implementarse antes que las secundarias. Por otro lado, si queremos resaltar algún aspecto positivo, considero que el proceso del diseño mejoró con respecto al primer proyecto ya que se hizo una separación mucho más clara de las responsabilidades de cada módulo del proyecto. Lo anterior también se logró gracias a que se aplicó a mayor profundidad la programación orientada a objetos ya que en este sistema todos los archivos fueron tratados como clases para poder instanciarlos como objetos. Otro punto mejorado con respecto al primer proyecto fue la mejora en cuanto al uso del patrón MVC además del uso del patrón DAO para la interacción de la base de datos.

En conclusión, este proyecto dejó un aprendizaje mayor del language de programación Rust y un primer acercamiento a las bases de datos y SQL. También dejó un gran aprendizaje en diseño y utilización de patrones de diseño, por lo cual, si bien en cuanto a la implementación de requerimientos del sistema el desarrollo del proyecto fue muy deficiente, en el aspecto del modelado hubo un gran avance en comparación al primer proyecto, por lo cual esta aplicación termina con un resultado bastante agri dulce.